

KV Database

基于 Rust 的教学型关系数据库设计与实现

课程项目小组

2026 年 7 月 16 日

摘要

本文报告一个使用 Rust 2024 实现的教学型关系数据库。系统以 4 KiB 磁盘页为基础，贯通 superblock、空闲页管理、slotted page、B+Tree、缓冲池、事务与 MVCC、SQL 处理、MySQL Wire Protocol、HTTP API 和 React 工作台。2026 年 7 月 16 日的复现实验中，74 个 Rust 测试和 87 项端到端协议/持久化测试全部通过，前端生产构建成功。项目参考 SQLite、PostgreSQL、MySQL InnoDB 和 CMU BusTub 的公开设计资料，但代码为小组独立实现；报告同时说明了结构差异、测试对比和当前限制。

关键词：Rust；关系数据库；B+Tree；事务；MVCC；持久化

目录

1	项目名称	3
2	团队成员与分工	3
3	项目背景与目标	3
3.1	项目背景	3
3.2	拟解决的问题	3
3.3	预期功能与完成范围	4
4	系统设计与实现思路	4
4.1	总体架构	4
4.2	关键数据结构	5
4.3	核心算法	5
4.4	关键代码位置	5

5	实验结果	6
5.1	实验环境与方法	6
5.2	功能与测试结果	7
5.3	命令行示例	7
5.4	结果分析与局限	7
6	开源项目参考说明	8
6.1	参考来源与引用方式	8
6.2	与原项目的区别、修改和改进	9

1 项目名称

项目名称：KV Database——基于 Rust 的教学型关系数据库。

项目采用 Cargo workspace 组织六个核心 crate，并提供 MySQL TCP 接口和 Web 工作台。其定位不是对成熟数据库做功能复刻，而是以可运行、可观察、可测试的规模展示一条完整链路：SQL 输入经过词法、语法、计划和执行阶段，进入事务层与 B+Tree 存储，最终落入磁盘页。

2 团队成员与分工

仓库提交历史能够识别到 hername111、huiyuhu 和 Vandy0915 三个 Git 署名，但仓库中没有可核验的真实姓名、学号与最终分工记录。为避免将昵称或邮箱误写为个人信息，表 1 保留正式填报位置；提交者必须在最终提交前按实际情况补齐或删除多余行。

表 1: 团队成员与主要分工（个人信息待提交者核实）

姓名	学号	主要分工
未提供	未提供	存储层、事务层、测试等实际负责内容待核实
未提供	未提供	SQL、网络协议、服务端等实际负责内容待核实
未提供	未提供	Web 工作台、工程化、文档等实际负责内容待核实

建议按代码提交、开发记录和演示内容填写分工，不把“共同完成”作为唯一描述。若实际成员少于三人，应删除多余行；若多于三人，应按相同格式增加。

3 项目背景与目标

3.1 项目背景

数据库系统同时涉及数据结构、文件系统、网络协议、并发控制和编程语言。直接调用现有数据库只能看到接口结果，难以观察 SQL 如何变成执行计划、记录如何组织为页面、事务提交如何影响可见性以及重启后元数据如何恢复。因此，本项目选择从零实现规模受控的数据库内核，以 Rust 的所有权、trait、错误传播和线程安全机制串联课程知识。

3.2 拟解决的问题

项目重点解决以下问题：一是将变长记录可靠地编码到固定大小页面，并拒绝越界或损坏数据；二是通过 B+Tree 和缓冲池完成有序存取及页面复用；三是建立从 SQL 到事务写集再到磁盘的执行路径；四是通过 MySQL 协议和 Web 界面提供可验证入口；五是用自动化测试覆盖正常功能、输入边界和重启持久化。

3.3 预期功能与完成范围

表 2: 目标功能与当前完成情况

模块	目标与已实现功能	状态
存储	4 KiB 页面、superblock、slotted page、空闲页链表、缓冲池、B+Tree、catalog	完成
SQL	DDL/DML、投影、条件表达式、排序、等值 JOIN、Lexer/Parser/Planner/Executor	完成
事务	BEGIN/COMMIT/ROLLBACK、写缓冲、读己之写、MVCC 可见性、表级锁	完成
协议	MySQL 握手、命令包、OK/ERR、结果集、TCP 分片重组与大小限制	完成
展示	HTTP 状态/查询/重置接口和 React 工作台	完成
恢复	catalog 与记录跨正常重启持久化	完成
崩溃恢复	WAL、checkpoint、崩溃原子恢复	未实现

4 系统设计与实现思路

4.1 总体架构

```

MySQL client --> network --+
                        +--> SQL --> txn --> storage --> kv.db
React UI -----> HTTP API ----+
```

1. **kv-common**: 共享类型、错误定义和 Pager trait。
2. **kv-storage**: 页面编解码、磁盘 Pager、LRU 缓冲、B+Tree、catalog。
3. **kv-txn**: 事务生命周期、读写集、版本链、可见性和表级锁。
4. **kv-sql**: Lexer、Parser、Planner 和 Executor。
5. **kv-network**: MySQL 握手、包解析、分片重组和结果集编码。
6. **kv-server**: 组装 TCP 服务与本地 HTTP 演示 API。

依赖保持单向：接口层调用 SQL 执行层，执行层依赖事务和存储，底层不依赖前端。该划分使各层可以独立单元测试，也让同一个执行器同时服务 MySQL 客户端和 Web 工作台。

4.2 关键数据结构

数据库文件由固定 4096 字节页面组成，page 0 为 superblock。其关键字段如表 3 所示。

表 3: superblock 关键字段

偏移	字段	作用
0-7	next_page_id	下一个未分配页号
8-15	free_list_head	空闲页单链表头
16-23	catalog_root	catalog B+Tree 根页
24-31	KVDBPAGE	文件魔数
32-35	version	格式版本，当前为 1

SlottedPage 的页头保存 page id、tuple 数量、free_start、free_end 和标志位。槽目录从页首向后增长，tuple 数据从页尾向前增长，槽项记录 tuple 的偏移和长度。validate_layout 检查槽目录边界、tuple 范围和重叠关系，使损坏数据返回 KvError 而不是触发越界访问。

4.3 核心算法

B+Tree。内部节点保存分隔键和子页号，叶节点保存有序键值并以 next 相连。插入先沿分隔键定位叶页，再按序插入；节点超容量时分裂并向父节点上推分隔键，根分裂时创建新根。叶链支持范围扫描。当前删除仅覆盖基础路径，尚未实现完整合并与再平衡。

页面分配与缓存。DiskPager 从空闲页链表优先分配页面，没有空闲页时扩展文件。打开文件时检查长度、魔数、版本、页号范围和链表环。BufferedPager 用 Mutex 保护 LRU 页面副本；释放页面时显式失效缓存，避免页号复用后读到旧内容。

SQL 执行。输入依次经过 Lexer、Parser、Planner 和 Executor。执行器读取 catalog 与 B+Tree，并在事务上下文中处理查询或写入。事务写操作先进入写缓冲与写集，提交时应用，回滚时丢弃；MVCC 版本链负责可见性判断，表级锁限制冲突访问。

协议与展示。MySQL 服务实现握手、命令包、OK/ERR 和结果集编码，可重组 TCP 分片，并拒绝超过 8 MiB 或截断的包。HTTP API 提供 GET /api/state、POST /api/query、POST /api/reset，请求体上限为 64 KiB。React 工作台从真实 API 获取 schema、记录、执行阶段和后端耗时，不在前端复制 SQL 规则。

4.4 关键代码位置

表 4: 实现模块与代码位置

文件	主要实现
crates/kv-storage/src/pager.rs	文件格式、superblock、空闲页链表和刷盘
crates/kv-storage/src/page.rs	Slotted Page 编解码和边界验证
crates/kv-storage/src/btree.rs	节点查找、插入、分裂、删除和扫描
crates/kv-storage/src/buffer.rs	LRU 缓冲池和缓存失效
crates/kv-txn/src/	事务管理、MVCC 与锁
crates/kv-sql/src/	SQL 词法、语法、计划与执行
crates/kv-network/src/protocol.rs	MySQL 包解析和结果编码
crates/kv-server/src/demo_http.rs	HTTP 路由、JSON 和请求限制
demo-client/src/App.jsx	工作台状态、历史与展示逻辑

5 实验结果

5.1 实验环境与方法

本次验证日期为 2026 年 7 月 16 日，操作系统为 Windows，测试对象为当前 Cargo workspace 和 React/Vite 前端。使用以下命令复现实验：

```
cargo fmt --check --all
cargo test --workspace --all-targets
cd demo-client && npm run build && cd ..
python test_protocol.py
```

test_protocol.py 启动隔离的服务器和数据目录，经 MySQL 协议执行 DDL、DML、条件查询、排序、JOIN、索引、事务和错误输入；随后两次重启服务，验证数据、元数据和删表状态的持久化。

5.2 功能与测试结果

表 5: 实测结果汇总

实验项	实际结果	结论
Rust 测试	8 个测试目标共 74 项，74 项通过、0 失败	通过
MySQL 连接	握手、认证、分片重组、截断包和超大包处理通过	通过
SQL 功能	建表/删表、增删改查、NULL、条件、排序、等值 JOIN、索引通过	通过
事务	读己之写、提交、回滚、UPDATE+DELETE 组合路径通过	通过
持久化	两次重启后数据、元数据和已删表状态均正确	通过
端到端测试	87 项全部通过、0 失败	通过
前端构建	Vite 7.3.5 转换 1579 个模块，2.23 秒完成生产构建	通过

Rust 测试的分布为：kv-common 5 项、kv-network 10 项、kv-server 3 项、兼容性测试 2 项、集成测试 3 项、kv-sql 20 项、kv-storage 23 项、kv-txn 8 项。存储测试中特别覆盖截断数据库文件、损坏槽目录、B+Tree 损坏页、过大记录、释放页缓存失效和空闲链表跨重启恢复。

5.3 命令行示例

端到端脚本的关键输出如下，可作为无需截图的可复现实验证据：

```
[31] PASS JOIN ... ON table.col = table.col
[46] PASS Row visible inside txn (write buffer)
[57] PASS ROLLBACK (undo DELETE)
[85] PASS After restart: data + metadata persisted
[87] PASS After restart: dropped table stays dropped
Total: 87, 87 passed
```

5.4 结果分析与局限

实验表明，项目已完成从协议输入到磁盘持久化的最小闭环，且正常功能和多类边界条件能够稳定返回预期结果。与只运行单元测试相比，87 项端到端测试额外覆盖真实 TCP 连接、服务器重启和文件复用，因此对模块集成与持久化结论提供了更强证据。

但“正常重启后可恢复”不等于“崩溃安全”。项目没有 WAL、checkpoint 和崩溃原子提交；B+Tree 未实现完整删除再平衡；MVCC 版本主要位于内存；锁粒度为表级；

MySQL 协议仅覆盖项目 SQL 子集。因此本实验只能得出当前功能范围内正确、可复现的结论，不能宣称达到生产数据库的 ACID、并发度或兼容性。

6 开源项目参考说明

本项目借鉴公开架构思想和数据布局原则，未把上游源码文件复制进仓库。引用均固定到版本或提交，并说明许可证、对应实现和差异。

6.1 参考来源与引用方式

表 6: 开源参考来源

项目	固定版本	许可证
SQLite	3.50.4	Public Domain
PostgreSQL	REL_18_0	PostgreSQL License
MySQL InnoDB	mysql-8.4.0	GPLv2
CMU BusTub	f0d9e375...b48508	MIT

- [1] SQLite Database File Format: <https://www.sqlite.org/fileformat2.html>
- [2] SQLite src/btree.c: <https://github.com/sqlite/sqlite/blob/version-3.50.4/src/btree.c>
- [3] PostgreSQL Database Page Layout: <https://www.postgresql.org/docs/current/storage-page-layout.html>
- [4] PostgreSQL bufpage.h: https://github.com/postgres/postgres/blob/REL_18_0/src/include/storage/bufpage.h
- [5] MySQL InnoDB Index Types: <https://dev.mysql.com/doc/refman/8.4/en/innodb-index-types.html>
- [6] MySQL page0types.h: <https://github.com/mysql/mysql-server/blob/mysql-8.4.0/storage/innobase/include/page0types.h>
- [7] CMU BusTub: <https://github.com/cmu-db/bustub/tree/f0d9e3753482d45f2b5919da1873684600b48508>

SQLite 用于对照固定页、文件头和 free-list；PostgreSQL 用于对照 slotted page 的页头与槽目录；InnoDB 用于对照 B+Tree 索引页字段和叶节点组织；BusTub 用于对照教学数据库的模块边界和 BufferPoolManager 职责。报告引用项目名、固定版本、官方链接和许可证，不使用含糊的“参考某数据库”表述。

6.2 与原项目的区别、修改和改进

表 7: 参考系统与本项目的对比分析

维度	参考系统	本项目的区别与改进
文件与空闲页	SQLite 使用成熟文件头、trunk/leaf freelist、journal/WAL	使用更小的 superblock 与页内单链表；增加魔数、版本、截断、越界和链表环检查，并用跨重启测试验证复用
数据页	PostgreSQL 保存 LSN、checksum、special space、MVCC 信息	仅保留课程所需页头、槽项和 tuple；增加集中式布局校验，但不实现 WAL 和页内 MVCC 头
索引	InnoDB 支持聚簇/二级索引、高并发和恢复	实现固定阶 B+Tree、叶链扫描和超页拒绝；接口同时承载表、catalog 和二级索引，但不宣称生产级恢复
缓冲池	BusTub 使用 page guard、pin count、锁和 replacer	用 Rust Pager trait、Mutex 和 LRU 隔离缓存职责；释放页时强制失效缓存，并有针对性回归测试
系统入口	上游项目主要面向各自完整客户端或教学测试	同一执行器同时提供 MySQL 协议和 React 工作台，API 返回真实后端状态与耗时

对比可见，本项目没有追求与 SQLite、PostgreSQL 或 InnoDB 的性能和功能对等，而是缩小数据结构后补充严格边界检查、跨重启测试和双入口展示，使各层在课程规模内可运行、可观察、可验证。相应代价是缺少 WAL、复杂并发控制、完整索引维护和崩溃恢复，这些差异已在实验结论中明确限定。